COURSE PROJECT -- HEALTH CARE DATA ANALYTICS

Trustworthy Deep Learning System for Pneumonia Detection

A calibrated, explainable, and uncertainty-aware AI framework for reliable chest X-ray screening

---

COMPREHENSIVE TECHNICAL HANDBOOK

This document provides an end-to-end technical reference covering every module, class, function, training procedure, evaluation metric, and system architecture of the XAI pneumonia detection pipeline.

Team Members

Goutham Kumar . Adhithiyan C . Vaibhav Pk . Yadarth Sangem

Research prototype for academic purposes only.
Not intended for clinical deployment without formal validation.

TABLE OF CONTENTS

1. Project Overview & Clinical Motivation
2. System Architecture -- Four Safety Layers
3. Configuration & Hyperparameters (configs/config.py)
4. Dataset & Data Pipeline (datasets/)
5. Model Architecture (models/)
6. Training Pipeline (training/)
7. Calibration -- Temperature Scaling (evaluation/)
8. Explainability -- Grad-CAM & FocusScore (explainability/)
9. Uncertainty -- Monte Carlo Dropout (explainability/)
10. TrustEngine -- Safety Scoring (explainability/)
11. End-to-End Inference Pipeline (inference/)
12. Evaluation Suite (evaluation/)
13. Project Structure & Module Reference
14. Limitations & Future Work

1. Project Overview & Clinical Motivation

Pneumonia is a life-threatening infection that fills the air sacs of the lungs with fluid or pus. Chest radiographs (X-rays) remain the most accessible first-line screening tool globally. Deep learning models have demonstrated radiologist-level accuracy on this task, but raw accuracy alone is insufficient for safe clinical use.

NOTE: The core thesis: A model that is accurate but uninterpretable, overconfident, and cannot detect its own failures is NOT safe for healthcare. We add four verifiable safety layers on top of the base classifier.

For every chest X-ray, the system produces FOUR outputs instead of just a label:

- **Predicted class:** NORMAL or PNEUMONIA.
- **Calibrated confidence:** An honest probability reflecting real-world accuracy.
- **Visual explanation:** A Grad-CAM heatmap showing WHERE the model looked.
- **Trust assessment:** A numerical TrustScore, trust level (High/Medium/Low), and an OOD flag.

Dataset

Kaggle Chest X-Ray Pneumonia dataset (Guangzhou Women and Children's Medical Center). 5,863 paediatric frontal chest X-rays in two classes: NORMAL and PNEUMONIA. Split into train/val/test with zero leakage by prepare_dataset.py.

Training Split

~5,216 images

Test Split

~624 images

2. System Architecture -- Four Safety Layers

The pipeline is structured as a stack of four independently verifiable safety layers. They are combined by the TrustEngine using a MULTIPLICATIVE formula, so a failure in any single layer collapses the trust score.

Layer 1: Classification

CNN backbone (default: ResNet18) maps a 224x224 RGB X-ray to a single logit -> sigmoid -> P(Pneumonia). Decision threshold: 0.5.

Layer 2: Calibration (Probability Honesty)

Temperature Scaling post-processes the logit so 80% confidence means ~80% of such cases are truly positive.

Layer 3: Explainability (Grad-CAM + FocusScore)

Grad-CAM produces a spatial heatmap. FocusScore measures whether attention falls inside the lungs.

Layer 4: Uncertainty (Monte Carlo Dropout)

30 stochastic forward passes with Dropout ON. High variance = model is guessing or image is out-of-distribution.

The Multiplicative Trust Formula:

```
TrustScore = Calibrated_Confidence
           x FocusScore
           x (1 - Uncertainty)
           x AgreementScore

All terms clipped to [0, 1]. A single failed check collapses TrustScore.
```

3. Configuration & Hyperparameters (configs/config.py)

All paths, model choices, and training hyperparameters are declared in one central file. Changing a value here propagates automatically to every module in the pipeline.

configs/config.py -- key constants

```
MODEL_TYPE         = 'resnet18'    # backbone: custom/resnet18/vgg16/densenet121/efficientnet_b0
IMAGE_SIZE         = 224           # input px (width = height)
BATCH_SIZE         = 32
NUM_EPOCHS         = 15
LEARNING_RATE      = 1e-4          # Adam optimizer lr
NUM_WORKERS        = 0             # DataLoader workers
DEFAULT_MC_SAMPLES = 30            # MC Dropout stochastic passes
DEFAULT_ECE_BINS   = 10            # calibration histogram bins
HEATMAP_THRESHOLD  = 0.4           # Grad-CAM binarisation threshold
MODEL_WEIGHTS_NAME = 'chestxray_best.pt'
CALIBRATOR_NAME    = 'temperature_scaling.pt'
```

ensure_dirs()

Called at every script entry-point. Creates MODEL_SAVE_DIR, OUTPUT_DIR, FIGURES_DIR, LOGS_DIR, PREDICTIONS_DIR, and REPORTS_DIR with mkdir(parents=True, exist_ok=True).

4. Dataset & Data Pipeline (datasets/)

ChestXRayDataset (datasets/dataset.py)

Subclasses torch.utils.data.Dataset. Walks data/train, data/val, or data/test looking for NORMAL/ and PNEUMONIA/ subdirectories. Stores (path, label) tuples.

ChestXRayDataset interface

```
CLASS_NAMES  = ['NORMAL', 'PNEUMONIA']   labels: 0, 1

__init__(root, transform, extension=('.jpg','.jpeg','.png'))
  _load_samples()  -- walks class folders, appends (path, int_label)
  __len__()        -- len(self.samples)
  __getitem__(i)   -- opens PIL image, applies transform, returns (tensor, label, path)
```

get_transforms() (datasets/transforms.py)

Returns torchvision transform pipelines per split:

- **Training:** RandomHorizontalFlip + RandomRotation(10deg) + Resize(224) + ToTensor + Normalize(ImageNet)
- **Val/Test:** Resize(224) + CenterCrop(224) + ToTensor + Normalize(ImageNet)

ImageNet mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]

get_dataloaders() (datasets/preprocessing.py)

Factory wrapping ChestXRayDataset in DataLoader objects. Training: shuffle=True. Val/Test: shuffle=False. Optional max_samples truncates splits for quick demos.

split_kaggle_dataset() (datasets/split_dataset.py)

Copies raw Kaggle images into train/val/test at a 70-15-15 stratified split. Seeded (default 42) for reproducibility. Prevents train/val leakage.

5. Model Architecture (models/)

Five backbone architectures, all sharing the same interface: single logit output, sigmoid activation, Dropout(0.5) before the classification head (required for MC Dropout).

5.1 ChestXRayCNN -- Custom CNN (models/cnn_model.py)

ChestXRayCNN forward pass

```
Input:  (B, 3, 224, 224)
Block1: Conv(3->32, 3x3) + BN + ReLU + MaxPool2d  -> (B, 32, 112, 112)
Block2: Conv(32->64)     + BN + ReLU + MaxPool2d  -> (B, 64, 56, 56)
Block3: Conv(64->128)    + BN + ReLU + MaxPool2d  -> (B, 128, 28, 28)
Block4: Conv(128->256)   + BN + ReLU + MaxPool2d  -> (B, 256, 14, 14)
Block5: Conv(256->512)   + BN + ReLU + MaxPool2d  -> (B, 512, 7, 7)
AdaptiveAvgPool2d(1,1)                            -> (B, 512, 1, 1)
Head:  Flatten -> Linear(512,256) -> ReLU -> Dropout(0.5) -> Linear(256,1)
Output: (B,1) logit  [sigmoid -> P(Pneumonia)]
```

5.2 ResNet18 (DEFAULT) -- get_resnet18()

Pre-trained ImageNet weights (IMAGENET1K_V1). Original fc(512->1000) replaced with Dropout(0.5) -> Linear(512, 1). Grad-CAM target: model.layer4[-1].conv2.

ResNet18 head replacement

```
model.fc = nn.Sequential(
    nn.Dropout(p=0.5),
    nn.Linear(512, 1)
)   # num_classes=1 -> sigmoid for binary
```

5.3 VGG16 -- get_vgg16()

Pre-trained. classifier[6] replaced: Linear(4096,1000) -> Linear(4096,1). Grad-CAM target: last Conv2d in model.features.

5.4 DenseNet121 -- get_densenet121()

Pre-trained. classifier replaced: Dropout(0.5) -> Linear(1024,1). Grad-CAM target: model.features.denseblock4.denselayer16.conv2.

5.5 EfficientNet-B0 -- get_efficientnet_b0()

Pre-trained. classifier[1] replaced: Linear(1280,1). Grad-CAM target: model.features[-1][0].

5.6 build_model() -- Factory function

Accepts model_type string, pretrained bool, num_classes int. Returns the correct nn.Module. All models output a single logit for binary classification.

5.7 get_last_conv_layer_for_model()

Given a model + model_type, returns the exact nn.Module for Grad-CAM hook registration. Each architecture has a different layer path to the final spatial convolutional layer.

5.8 Ensemble Agreement (models/ensemble.py)

models/ensemble.py interface

```
majority_vote(class_indices) -> (majority_class, agreement_score)
  agreement_score = majority_count / total_models
  Range: [0.5, 1.0] for binary. 1.0 means unanimous agreement.

agreement_from_probabilities(probs, threshold=0.5) -> (class, score)
  Threshold each P(Pneumonia) -> binary vote -> calls majority_vote()
```

6. Training Pipeline (training/)

Loss Function

nn.BCEWithLogitsLoss (Binary Cross-Entropy with Logits). Combines sigmoid + BCE in one numerically stable operation. Raw logit in, no sigmoid applied DURING training.

BCE with Logits formula

```
Loss = -[ y*log(sigmoid(z)) + (1-y)*log(1-sigmoid(z)) ]
where z = logit, y in {0=NORMAL, 1=PNEUMONIA}
```

Optimiser

Adam with lr=1e-4, beta1=0.9, beta2=0.999, eps=1e-8. Chosen for fast convergence on medical imaging and robustness to sparse gradients.

train_one_epoch() (training/trainer.py)

train_one_epoch() pseudocode

```
model.train()   # enables BN updates + Dropout
for images, labels, _ in loader:
    optimizer.zero_grad()
    logits = model(images).squeeze(-1)
    loss = BCEWithLogitsLoss(logits, labels.float())
    loss.backward()
    optimizer.step()
    probs = sigmoid(logits.detach())
    preds = (probs >= 0.5).long()
    accumulate loss + correct count
return avg_loss, accuracy
```

evaluate_epoch() (training/trainer.py)

Identical loop with model.eval() and @torch.no_grad(). No weight updates. Returns avg_loss, accuracy on validation set.

main() (training/train.py) -- Full Training Orchestration

- **1. Parse CLI args:** --model_type, --epochs, --batch_size, --lr, --seed, --quick.
- **2. set_seed(42):** Seeds Python random, NumPy, PyTorch, and CUDA for reproducibility.
- **3. Build DataLoaders:** get_dataloaders() for train/val splits.
- **4. Build model:** build_model(model_type, pretrained=True, num_classes=1).
- **5. Training loop:** train_one_epoch -> evaluate_epoch -> save if val_acc improves.
- **6. Best checkpoint:** Saved as checkpoints/chestxray_best.pt with model_state_dict + model_type.
- **7. model_config.txt:** Records model_type for inference-time backbone selection.
- **8. Plots:** training_curves.png (loss + accuracy vs epoch) saved to outputs/.

NOTE: --quick flag: sets epochs=2 and max_samples=128 per split. CPU-friendly demo that completes in

under a minute. Full training (15 epochs) recommended on GPU.

7. Calibration -- Temperature Scaling (evaluation/calibration.py)

Standard neural networks are overconfident: they output 0.97 confidence even when essentially guessing. Temperature Scaling is a post-hoc method that aligns predicted confidence with empirical accuracy WITHOUT changing the class ranking or re-training.

The Formula

Temperature Scaling formula

```
p_calibrated = sigmoid(logit / T)

T > 1 : softens overconfident predictions  (flattens sigmoid curve)
T < 1 : sharpens underconfident predictions (steepens sigmoid curve)
T = 1 : no change

Accuracy unchanged because relative ranking of logits is preserved.
```

TemperatureScaling class

TemperatureScaling class interface

```
temperature = nn.Parameter(torch.tensor(1.5))  # initial T, learnable

forward(logits)         -> logits / clamp(T, min=1e-4)
calibrate_logits()      -> same as forward
calibrate_probs(logits)-> sigmoid(logits / T)   # for binary
property T              -> float value of current temperature
```

fit_temperature() -- How T is Learned

The CNN is completely FROZEN. Only the scalar T is optimised.

- **Step 1:** Run entire validation set through frozen model, collect all logits + labels.
- **Step 2:** optimizer = L-BFGS([temperature.temperature], lr=0.01, max_iter=50).
- **Step 3:** Closure: loss = BCELoss(sigmoid(logits/T), labels). Call optimizer.step(closure).
- **Returns:** Fitted TemperatureScaling module saved to checkpoints/temperature_scaling.pt.

L-BFGS is chosen because this is a 1-parameter problem and converges in very few iterations.

Expected Calibration Error (ECE)

ECE formula (evaluation/reliability.py: compute_ece)

```
ECE = sum over bins b:
    |b| / N  *  |mean_confidence(b) - mean_accuracy(b)|

n_bins = 10 (equal-width bins from 0 to 1)
Lower is better. ECE = 0 means perfect calibration.
We report ECE before and after temperature scaling to demonstrate improvement.
```

8. Explainability -- Grad-CAM & FocusScore (explainability/)

8.1 Grad-CAM Algorithm (explainability/gradcam.py)

Gradient-weighted Class Activation Mapping (Selvaraju et al., 2017). Produces a coarse spatial heatmap showing which regions drove the predicted class score.

Grad-CAM computation -- GradCAM.generate()

```
Step 1: Forward pass through model
        Hook captures activation maps A^k at target_layer  (C x H' x W')

Step 2: Backward pass of class score
        Hook captures gradients dY/dA^k at target_layer

Step 3: Channel weights (global average pooled gradients):
        alpha^k = (1/H'W') * sum_{i,j}  dY/dA^k_{ij}

Step 4: Weighted sum + ReLU (only positive contributions):
        CAM = ReLU( sum_k  alpha^k * A^k )

Step 5: cv2.resize to 224x224,  min-max normalise to [0, 1]
        Returns np.float32 array (H, W)
```

>> GradCAM class methods

GradCAM class full interface

```
__init__(model, target_layer)     -- stores refs, initialises hook list
_save_activation(module,in,out)   -- forward hook: saves output.detach()
_save_gradient(module,gi,go)      -- backward hook: saves go[0].detach()
_register_hooks()                 -- attaches forward + full_backward hooks
_remove_hooks()                   -- removes all hooks after generation
generate(input_tensor, target_class=None) -> np.ndarray (H,W) in [0,1]

generate_gradcam_heatmap()        -- functional (stateless) API wrapper
overlay_heatmap(heatmap, img)     -- cv2 jet colormap blend, alpha=0.5
```

>> Binary vs. Multi-class Grad-CAM

Binary (single logit): target_class=1 -> backward on output.sum(). target_class=0 -> backward on (-output).sum(). Multi-class (2 outputs): one-hot mask selects the target class score before backprop.

8.2 FocusScore (explainability/attention_maps.py)

Turns the qualitative heatmap into a quantifiable constraint. A model can produce a visually plausible heatmap that focuses on the wrong anatomy. FocusScore penalises this.

focus_overlap_score() formula

```
FocusScore = Area(GradCAM_binary AND LungMask) / Area(GradCAM_binary)

GradCAM_binary = (heatmap >= HEATMAP_THRESHOLD)   [HEATMAP_THRESHOLD=0.4]
LungMask       = binary mask of lung region
```

```
Range [0, 1]:
  ~ 1.0  -> attention fully inside lungs  (good)
  ~ 0.0  -> attention almost entirely outside lungs  (dangerous shortcut)
```

>> Lung Mask Sources

- **Geometric Prior (default):** Central rectangle y=[15%,90%], x=[10%,90%] of image. Approximates typical frontal CXR lung field position.
- **SimpleUNet (optional):** Lightweight 3-encoder/3-decoder U-Net. Can be loaded from a pretrained checkpoint (e.g. trained on JSRT/Montgomery). Falls back to geometric prior automatically if no checkpoint is supplied.

>> SimpleUNet Architecture

SimpleUNet architecture (attention_maps.py)

```
Encoder:    enc1(in->32) -> pool -> enc2(32->64) -> pool -> enc3(64->128) -> pool
Bottleneck: (128->256)
Decoder:    up3 + concat(enc3) -> dec3(256->128)
            up2 + concat(enc2) -> dec2(128->64)
            up1 + concat(enc1) -> dec1(64->32)
Output:     Conv2d(32->1, kernel=1) -> sigmoid -> threshold @ 0.5
Each block: Conv(3x3)+BN+ReLU repeated twice
```

9. Uncertainty -- Monte Carlo Dropout (explainability/uncertainty.py)

Reference: Gal & Ghahramani (2016), 'Dropout as a Bayesian Approximation' (arXiv:1506.02142). Dropout at inference time approximates sampling from a posterior distribution over model weights. Variance of samples is a proxy for epistemic (model) uncertainty.

enable_dropout() / disable_dropout()

enable_dropout(): iterates model.modules(), calls .train() on every nn.Dropout and nn.Dropout2d -- re-enables stochasticity while leaving BatchNorm in eval mode (preserving running statistics). Call AFTER model.eval().

disable_dropout(): calls .eval() on Dropout layers -- fully deterministic mode.

predict_with_uncertainty() -- Core Function

predict_with_uncertainty() -- exact procedure

```
predict_with_uncertainty(model, image_tensor, n_samples=30, device):

  1. model.eval()          # freeze BN, base dropout state
  2. enable_dropout(model) # re-enable ONLY dropout layers
  3. with torch.no_grad(): # no gradient computation needed
        for _ in range(n_samples):
            logits = model(image_tensor)
            prob   = sigmoid(logits).item()
            probabilities.append(prob)
  4. model.eval()          # fully disable dropout
  5. samples = np.array(probabilities)
     return {
       'mean_prediction': mean(samples),   # raw confidence
       'variance':        var(samples),    # THE uncertainty score
       'std':             std(samples),
       'samples':         samples           # (n_samples,) array
     }
```

Interpreting Variance

- **variance < 0.01 (Low):** Consistent predictions. Model is confident and stable.
- **0.01 <= variance < 0.05 (Medium):** Some variation. Case may be ambiguous. Human review recommended.
- **variance >= 0.05 (High):** Model is essentially guessing. Output must NOT be trusted. OOD highly likely.

MCDropoutPredictor class

Stateful wrapper: stores model, n_samples, and device. predict(image_tensor) delegates to predict_with_uncertainty(). uncertainty_level(variance) returns 'low', 'medium', or 'high' label.

ood_flag_from_confidence_uncertainty()

Delegates to TrustEngine.detect_ood(). Returns True (UNRELIABLE) if calibrated_confidence < 0.6 AND variance > 0.03.

10. TrustEngine -- Safety Scoring (explainability/trustscore.py)

The TrustEngine is the unifying safety arbiter. It combines all signals into a single scalar TrustScore and produces a human-readable trust level.

compute_trust_score_with_uncertainty() -- Current Formula

compute_trust_score_with_uncertainty()

```
confidence_component = max(p, 1-p)  # reward confidence in EITHER class
trust = confidence_component
      * clip(focus_score, 0, 1)
      * (1 - clip(uncertainty, 0, 1))
return clip(trust, 0, 1)

Why multiplicative: a single failed check (focus=0.1 or uncertainty=0.9)
collapses the score. Addition would let high confidence hide bad focus.
```

TrustEngine.compute_trust_score() -- Full Formula

TrustEngine.compute_trust_score() -- adds AgreementScore

```
trust = max(p, 1-p)
      * clip(focus_score, 0, 1)
      * (1 - clip(uncertainty, 0, 1))
      * clip(agreement_score, 0, 1)    # adds ensemble signal
return clip(trust, 0, 1)
```

get_trust_level() -- Decision Logic

Trust level decision tree

```
if uncertainty > 0.3 or focus_score < 0.35:   return 'low'
if confidence > 0.8 and uncertainty < 0.1:    return 'high'
if 0.6 <= confidence <= 0.8
   or 0.1 <= uncertainty <= 0.3:              return 'medium'
else:                                          return 'medium'
```

TrustEngine.detect_ood() -- OOD / UNRELIABLE Flag

OOD detection rule

```
TrustEngineConfig: confidence_threshold=0.6, uncertainty_threshold=0.03

detect_ood(calibrated_confidence, uncertainty):
    return (calibrated_confidence < 0.6) AND (uncertainty > 0.03)
```

This is a conservative heuristic gate, not a learned OOD model. It catches low-confidence + high-variance cases -- typical of inputs the CNN has never seen in training (blurry scans, non-CXR images, extreme noise).

summarize() -- All-in-one Convenience

TrustEngine.summarize()

```
engine.summarize(calibrated_confidence, focus_score, uncertainty, agreement_score)
    -> {'trust_score': float, 'ood_flag': bool}
```

11. End-to-End Inference Pipeline (inference/)

run_pipeline() is the master integration function. It sequentially applies all four safety layers and returns a rich output dictionary plus a 4-panel figure.

load_and_transform_image()

Image loading and normalisation

```
load_and_transform_image(image_path, device):
    img_pil = Image.open(path).convert('RGB')
    transform = Resize(224) + ToTensor + Normalize(ImageNet_mean, ImageNet_std)
    img_tensor = transform(img_pil).unsqueeze(0).to(device)  # (1,3,224,224)
    img_np = denormalize(tensor) * 255  # uint8 RGB for visualisation
    return img_tensor, img_np
```

run_pipeline() -- Step-by-Step

- **Step 1 -- Setup:** ensure_dirs(). Detect CUDA/CPU. load_model_and_calibrator() from checkpoints/. get_last_conv_layer_for_model() for Grad-CAM target.
- **Step 2 -- Load image:** load_and_transform_image() -> img_tensor (1,3,224,224) + img_np (RGB uint8).
- **Step 3 -- MC Dropout:** predict_with_uncertainty(model, img_tensor, n_samples=30). Returns mean_prediction, variance, std, samples array.
- **Step 4 -- Calibrated confidence:** Deterministic forward pass (model.eval(), no_grad). calibrator.calibrate_probs(logits) -> calibrated_prob.
- **Step 5 -- Grad-CAM:** GradCAM(model, target_layer).generate(img_tensor, target_class). overlay_heatmap(heatmap, img_np, alpha=0.5) -> RGB overlay.
- **Step 6 -- FocusScore:** LungSegmenter.segment(img_tensor) -> lung_mask. focus_overlap_score(heatmap, lung_mask, threshold=0.4) -> focus_score.
- **Step 7 -- Ensemble:** If auxiliary DenseNet/EfficientNet checkpoints are supplied, run inference, append to ensemble_probs. agreement_from_probabilities() -> agreement_score.
- **Step 8 -- TrustEngine:** engine.summarize(calibrated_prob, focus_score, variance, agreement_score). get_trust_level() -> 'High', 'Medium', or 'Low'.
- **Step 9 -- Output:** Returns dict with all metrics. build_inference_figure() creates a 4-panel plot (original X-ray | Grad-CAM overlay | MC histogram | metrics summary). Saved to outputs/<name>_uncertainty_gradcam.png.

Output Dictionary Keys

run_pipeline() return dictionary

```
{
  'predicted_class':        'NORMAL' or 'PNEUMONIA',
  'raw_confidence':          float  (MC Dropout mean),
  'calibrated_confidence':   float  (temperature-scaled),
  'uncertainty_score':       float  (MC Dropout variance),
  'focus_score':             float  (Grad-CAM x lung overlap),
  'agreement_score':         float  (ensemble majority fraction),
  'trust_score':             float  [0,1],
  'trust_level':             'High' / 'Medium' / 'Low',
  'ood_flag':                bool   (True = UNRELIABLE),
```

```
    'overlay_path':            str    (path to saved figure)
}
```

12. Evaluation Suite (evaluation/)

12.1 Classification Metrics (evaluation/metrics.py)

- **Accuracy:** Fraction of correct predictions = (TP+TN) / N.
- **Precision:** TP / (TP+FP). Fraction of predicted-positive that are true-positive.
- **Recall (Sensitivity):** TP / (TP+FN). Fraction of true-positive cases detected.
- **F1-Score:** 2 * Precision * Recall / (Precision + Recall). Handles class imbalance.
- **Confusion Matrix:** 2x2 [[TN,FP],[FN,TP]] via sklearn.

12.2 Calibration Metrics (evaluation/reliability.py)

- **compute_ece(probs, labels, n_bins=10):** Expected Calibration Error. Reported before and after temperature scaling.
- **reliability_diagram():** Bar chart of accuracy (blue bars) vs confidence (red line) per bin. A well-calibrated model hugs the diagonal. Returns (ECE, bin_accs, bin_confs).

12.3 Full Evaluation (evaluation/evaluate.py)

- **Standard pass:** Accuracy, Precision, Recall, F1, confusion matrix on test set.
- **Calibration:** ECE raw vs calibrated. Plots reliability diagram.
- **--trust_eval flag:** Per-image MC Dropout (20 samples), Grad-CAM, FocusScore, TrustEngine. Reports mean/std FocusScore, mean/std TrustScore, and OOD detection accuracy.
- **Outputs:** evaluation_results.png (CM + reliability), trustscore_distribution.png.

12.4 Faculty Report (evaluation/faculty_report.py)

Generates a 12-panel figure pack including: training curves, confusion matrix, reliability diagrams (before/after calibration), Grad-CAM gallery, FocusScore distribution, TrustScore distribution, uncertainty histogram, and MC sample traces.

13. Project Structure & Module Reference

Complete project directory structure

```
project_root/
+-- configs/
|   +-- config.py              Central hyperparameters and paths
+-- datasets/
|   +-- dataset.py             ChestXRayDataset (PyTorch Dataset)
|   +-- preprocessing.py       get_dataloaders() factory
|   +-- transforms.py          get_transforms() with augmentation
|   +-- split_dataset.py       Train/val/test splitting utilities
+-- models/
|   +-- cnn_model.py           All 5 backbones + build_model() factory
|   +-- ensemble.py            majority_vote() + agreement_from_probabilities()
+-- training/
|   +-- train.py               Training CLI entry point
|   +-- trainer.py             train_one_epoch() + evaluate_epoch()
+-- evaluation/
|   +-- calibrate.py           Fit temperature scalar (CLI)
|   +-- calibration.py         TemperatureScaling class + fit_temperature()
|   +-- evaluate.py            Full test evaluation (CLI)
|   +-- metrics.py             Accuracy/Precision/Recall/F1
|   +-- reliability.py         compute_ece() + reliability_diagram()
|   +-- confusion_matrix.py    plot_confusion_matrix()
|   +-- faculty_report.py      12-panel figure pack
+-- explainability/
|   +-- gradcam.py             GradCAM class + overlay_heatmap()
|   +-- attention_maps.py      LungSegmenter + focus_overlap_score()
|   +-- uncertainty.py         predict_with_uncertainty() + MCDropoutPredictor
|   +-- trustscore.py          TrustEngine + TrustAnalyzer + get_trust_level()
|   +-- visualization.py       build_inference_figure() 4-panel plot
+-- inference/
|   +-- inference.py           run_pipeline() master function
|   +-- predict.py             CLI entry point
|   +-- api.py                 Programmatic API wrapper
+-- scripts/
|   +-- build_faculty_ppt.py   PowerPoint generator
|   +-- build_handbook.py      This PDF handbook generator
|   +-- demo.py                Faculty demo environment checker
|   +-- run_pipeline.py        Full pipeline runner
+-- utils/
|   +-- io.py                  load_model_and_calibrator()
|   +-- helpers.py             matplotlib backend utils
|   +-- seed.py                set_seed()
|   +-- logger.py             get_logger()
+-- tests/
|   +-- test_core.py           Unit tests (ResNet dropout, trust bounds)
|   +-- test_imports.py        Smoke tests for all imports
+-- notebooks/
|   +-- colab_faculty_demo.ipynb
+-- checkpoints/
|   +-- chestxray_best.pt      Best model weights
|   +-- temperature_scaling.pt  Fitted calibrator
+-- outputs/                   All generated figures and plots
```

```
+-- docs/                    Presentations and handbooks
```

14. Limitations & Future Work

> **NOTE:** IMPORTANT: This is a research prototype for academic purposes ONLY. It has not undergone clinical validation, reader studies, or regulatory review. It must NOT be used for real patient care decisions.

Known Limitations

- **Dataset bias:** Kaggle CXR is paediatric-only. Adult pathology, COVID-19, TB, and non-standard equipment will degrade performance significantly.
- **Geometric lung prior:** FocusScore uses a static rectangular mask unless a U-Net is provided. Not radiologist-grade segmentation.
- **OOD detection is a heuristic:** The (confidence<0.6 AND variance>0.03) rule can miss subtle OOD cases. Not a learned density estimator.
- **Single-model ensemble:** Without extra checkpoints, AgreementScore is always 1.0 -- no signal.
- **JPEG/PNG only:** Clinical systems use DICOM. No pydicom integration currently.
- **Shortcut learning:** Grad-CAM reveals shortcuts but does NOT prevent them during training.

Future Work

- **Real lung segmentation:** Train SimpleUNet on JSRT/Montgomery datasets.
- **DICOM/PACS integration:** Add pydicom support for clinical workflows.
- **Multi-class disease:** Extend to 14-class NIH ChestX-ray14.
- **Reader study:** Validate TrustScore utility with radiologists.
- **Dedicated OOD detector:** Replace heuristic with learned anomaly detection.
- **Larger ensembles:** Train 3-5 diverse architectures for meaningful agreement.